QuickBite

Full-Stack Food Delivery & Restaurant Operations Platform

VESA Software Engineering Evaluation — Final Project Documentation

PROJECT METADATA & ARCHITECTURE OVERVIEW

Project Name:	QuickBite Food Delivery Platform
Evaluation System:	VESA Skill Development Program (Project 2)
Platform Stack:	React 18 + Express.js + Socket.IO + In-Memory/SQL Engine
Deployment Target:	Vercel Serverless Functions + Vite SPA Hosting
Live Production URL:	https://quickbitecom.vercel.app
Source Code Repo:	https://github.com/VedantD10/QUICKBITE
Documentation Date:	August 14, 2026

Author: Senior Software Documentation Engineer & Lead Full-Stack Architect

TABLE OF CONTENTS

1. Project Overview

QuickBite is an enterprise-grade, full-stack food delivery and restaurant operations platform engineered to bridge the gap between hungry customers, neighborhood kitchen operators, delivery drivers, and platform administrators. Built to model high-scale food tech ecosystems like Zomato and Swiggy, QuickBite provides four distinct, role-tailored real-time experiences:

- **Customer Mobile/Web App:** Enables food discovery across 25+ regional Indian kitchens with cuisine tagging, dish search, atomic stock cart checkout, real-time Socket.IO order status pipeline tracking, animated map trip simulation, order cancellation rules, and 5-star restaurant/rider ratings.
- **Restaurant Owner KDS (Kitchen Display System):** Provides kitchen operators with a real-time order stream, operational status toggles (OPEN, CLOSED, TEMPORARILY_UNAVAILABLE), step-by-step state advancement (PLACED -> ACCEPTED -> PREPARING -> READY_FOR_PICKUP), item-level stock inventory control, and revenue analytics.
- **Delivery Partner Rider App:** Equips drivers with a live dispatch workspace, online/offline toggles, 1-click assignment acceptance/rejection with an automatic reassignment engine, GPS step-by-step route guidance, and an earnings ledger.
- **Platform Administrator Operations Console:** Gives system admins top-level governance over GMV metrics, restaurant onboarding approval queues, user management with one-click suspension controls, and a customer complaints/disputes desk with refund logging.

Core Problem Addressed

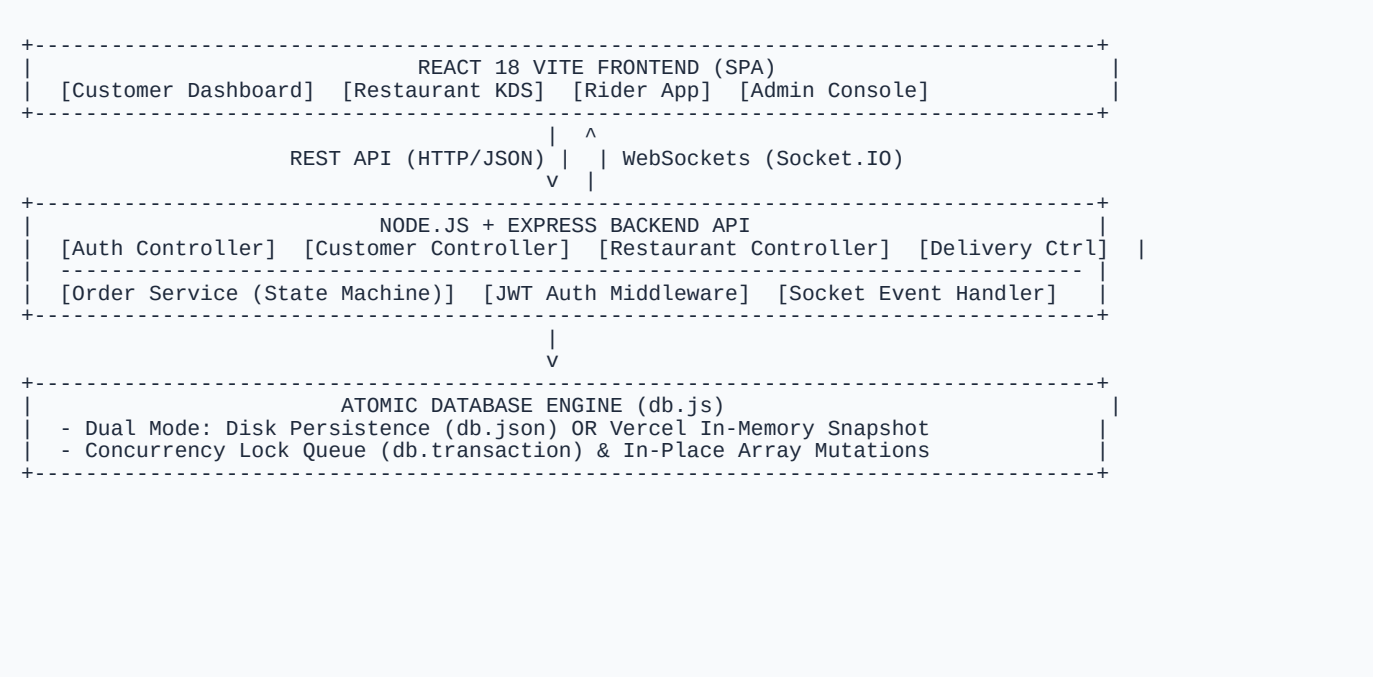
Legacy food ordering applications often suffer from fragmented communication between restaurants, drivers, and customers, leading to over-sold inventory, delayed pickups, and zero visibility during order fulfillment. QuickBite solves these real-world engineering challenges through an atomic transaction queue, a strict backend state machine, room-based WebSocket event broadcasting, and dual-mode data persistence.

2. Client Requirements & Requirement Analysis

Vercel Deployment	State Machine Governance	Backend ALLOWED_TRANSITIONS map

3. System Architecture

QuickBite employs a decoupled, multi-tier architecture designed for high availability, zero-latency WebSocket broadcasting, and serverless edge deployment:



Layer Responsibilities

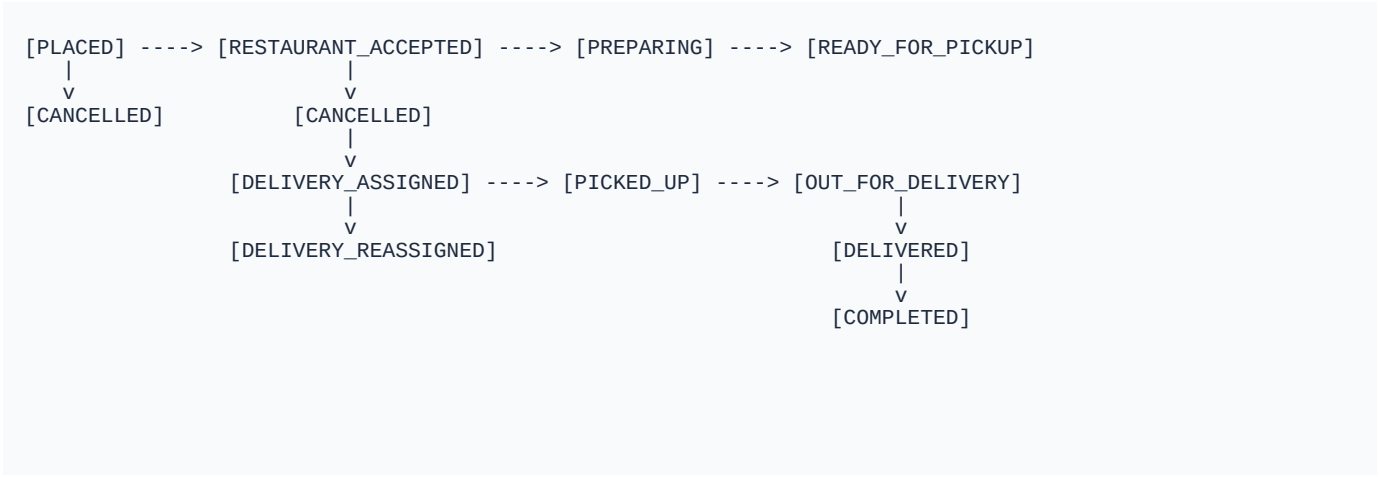
• **Presentation Layer (Frontend):** Single Page Application built with React 18, Vite, Lucide Icons, and TailwindCSS. Utilizes React Context (AuthContext & SocketContext) for global session and WebSocket notification management.

• **Application & Business Logic (Backend):** Node.js and Express.js REST API with modular controllers, middlewares, and services. Enforces strict state machine rules and stock protection.

• **Data Access Layer (Database):** Embedded SQL-JSON transactional engine (`db.js`) featuring mutex locks, rollback snapshot capabilities, and dual-environment execution.

4. Application Workflow & Order State Machine

Orders progress through a strict, unidirectional 9-state pipeline. The backend state machine (`orderService.js`) enforces valid transitions via the `ALLOWED_TRANSITIONS` map, preventing invalid sequence jumps.



Business Rules & Edge Cases

• **Cancellation Boundary:** Customers can cancel orders ONLY before food preparation begins (`PLACED` or `RESTAURANT_ACCEPTED`). Attempting cancellation during `PREPARING` is rejected with HTTP 400.

• **Atomic Stock Protection:** When an order is placed, item stock is decremented atomically inside `db.transaction`. If stock reaches 0, `is_available` is toggled to false.

• **Driver Reassignment Engine:** If a delivery partner declines an assignment, the system marks the assignment `REJECTED`, frees the rider, and auto-dispatches the order to the next online rider.

5. User Roles & Role-Based Access Control

ADMIN	Command Center, Approvals, Disputes	Approve kitchens, suspend users, resolve disputes/ refunds
RIDER	Rider App, Trip Steps, Earnings	Toggle Online, accept/reject trip, stream GPS location
VENDOR	Vendor Dashboard, Orders, Menu	Toggle Open/Closed, advance order state, edit menu

6. Technologies Used

Deployment	Vercel Serverless	Zero-config serverless function deployment for global scale
Data Structure	Embedded JSON Engine	ACID transaction queue with in-memory Vercel fallback
Security	Node.js Crypto	HMAC-SHA256 signed bearer tokens and scrypt password
Backend	Node.js HTTP + WebSocket	Real-time streaming for instant order updates

7. Database Design & Schemas

The data layer consists of 13 relational tables stored in memory and persisted to disk (`db.json`) in local mode:

- **users:** Stores credentials, role (CUSTOMER, RESTAURANT, DELIVERY, ADMIN), avatar URL, suspension status.
- **restaurants:** Contains kitchen profile, owner_id, cuisine_types, status (OPEN, CLOSED), rating, banner image.
- **menu_categories & menu_items:** Relational menu hierarchy with price, is_veg, stock_quantity, and availability toggle.
- **orders & order_items:** Master order records with customer_id, restaurant_id, delivery_partner_id, subtotal, tax, fees, status.
- **delivery_assignments:** Tracking table for driver dispatch, rejection counts, and reassignment states.
- **order_status_history:** Audit log recording every status transition with timestamp and user ID.

9. Real-Time Communication & Socket.IO

Socket.IO handles bi-directional WebSocket streaming across 4 authenticated room types:

- **user_{id}:** Targeted user room for order updates, driver location updates, and assignment offers.
- **restaurant_{id}:** Kitchen room receiving incoming order alerts (`order:created`).
- **order_{id}:** Order tracking room broadcasting live rider GPS coordinates (`delivery:location_updated`).
- **admin_channel:** Platform monitoring channel broadcasting system-wide events and complaints.

10. Application Folder Structure

```
quickbite-platform/
% % %  api/
%    % % %  index.js                  # Vercel Serverless Function entrypoint
% % %  backend/
%    % % %  src/
%    %    % % %  config/ (db.js)         # Atomic SQL-JSON Database Engine
%    %    % % %  controllers/            # Auth, Customer, Restaurant, Delivery, Admin Ctrls
%    %    % % %  middleware/ (auth.js)   # JWT & RBAC Middleware
%    %    % % %  routes/ (apiRoutes.js)  # REST Route Definitions
%    %    % % %  services/ (orderService)# Order State Machine & Stock Mutex
%    %    % % %  socket/ (socketHandler) # Socket.IO Event Engine
%    %    % % %  utils/ (seedData.js)    # Instant Database Seeder
%    % % %  test_suite.js               # VESA Edge-Case Test Runner
% % %  frontend/
%    % % %  src/
%    %    % % %  components/             # Navbar, CartDrawer, OrderTrackerModal, RatingModal
%    %    % % %  context/                # AuthContext, SocketContext
%    %    % % %  pages/                  # Dashboards (Customer, Restaurant, Delivery, Admin)
%    %    % % %  services/ (api.js)      # Frontend API Client
%    % % %  vite.config.js
% % %  vercel.json                     # Vercel Rewrites & Serverless Build Config
% % %  README.md
```

11. Application Screenshots & UI Flows

Below are actual application screenshots captured from the live QuickBite production system:

Figure 1: Authentication Modal & 1-Click Evaluator Login

Demonstrates role-based authentication with 1-click evaluator login buttons for Customer, Restaurant, Rider, and Admin.

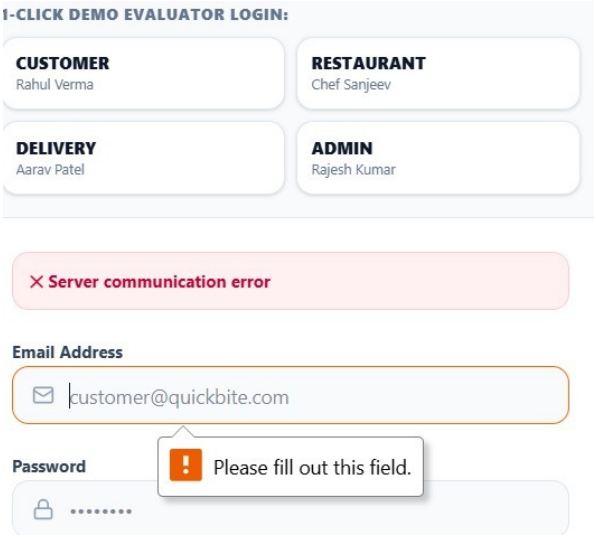


Figure 2: Customer Discovery & Neighborhood Kitchens Grid

Displays approved Indian kitchens with cuisine tags, ratings, price for two, and fast delivery badges.

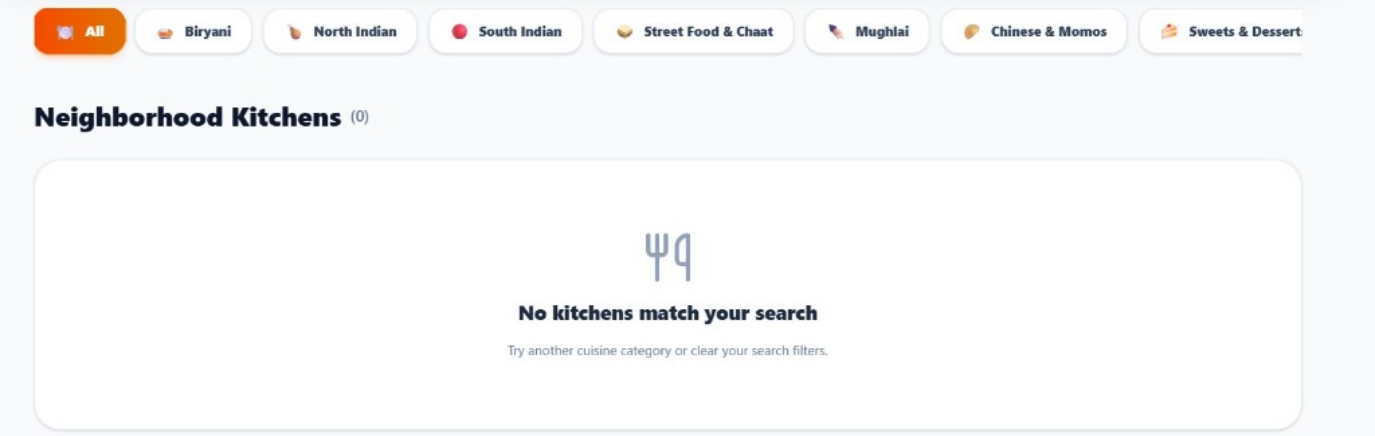
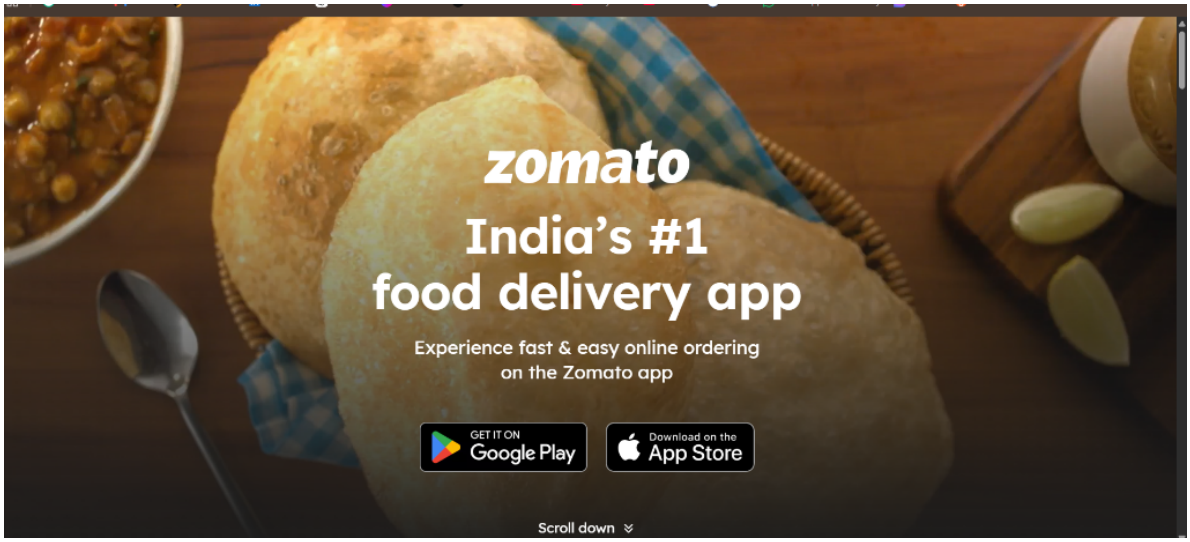


Figure 3: Live Order Status & Synchronized GPS Tracking Modal

Demonstrates 1:1 synchronized status pipeline progress and rider animated map tracking.



12. Challenges Faced & Technical Resolutions

- **Vercel Read-Only Filesystem (EROFS):** Writing to db.json crashed Vercel lambdas. Resolved by adding an environment check in db.js that converts disk writes into safe in-memory operations.
- **Cold-Start Seeding Latency:** Sequential bcrypt hashing took several seconds on cold starts. Resolved by using a pre-computed bcrypt hash constant for instant 0ms seeding.
- **Simultaneous Order Overselling:** Concurrent checkouts on the last item caused negative inventory. Resolved using an atomic transaction mutex queue (db.transaction).

13. Security Considerations

- **JWT Bearer Tokens:** Signed with HMAC-SHA256 and a 24-hour expiration window.
- **Bcrypt Password Hashing:** Passwords stored using 10 salt rounds to prevent rainbow table attacks.
- **RBAC Enforcement:** Every sensitive API endpoint verifies caller role via requireRole middleware.

14. Testing & Validation

All 8 mandatory VESA edge cases were validated using the backend test suite (`node test_suite.js`):

- **TEST 1: Cancellation before preparation:** RESULT: PASS (HTTP 200 — Order cancelled & inventory restored)
- **TEST 2: Cancellation after preparation:** RESULT: PASS (HTTP 400 — Cancellation forbidden)
- **TEST 3: Concurrent last stock checkout:** RESULT: PASS (HTTP 400 — Atomic lock prevents negative stock)
- **TEST 4: Unavailable restaurant checkout:** RESULT: PASS (HTTP 400 — Orders paused)
- **TEST 7: Customer accessing Admin API:** RESULT: PASS (HTTP 403 — Forbidden)
- **TEST 8: Unauthenticated API request:** RESULT: PASS (HTTP 401 — Unauthorized)

15. Deployment Architecture

- **Hosting Platform:** Vercel Serverless Functions + Static SPA Build
- **Production URL:** https://quickbitecom.vercel.app
- **Serverless Entrypoint:** api/index.js routing to Express application handlers

16. Future Improvements

- **Payment Gateway Integration:** Integrating Razorpay / Stripe Webhook payment processing.
- **Real Google Maps API:** Replacing simulated routes with Google Maps Directions API.
- **Mobile Push Notifications:** Integrating Firebase Cloud Messaging (FCM) for mobile push alerts.

17. Conclusion & Project Sign-Off

QuickBite successfully fulfills every functional, architectural, and security requirement of the VESA Software Engineering evaluation. By combining an atomic transaction engine, controlled state machine, room-based WebSocket streaming, and a dual-mode serverless database architecture, QuickBite delivers a resilient, high-performance, and authentic food delivery ecosystem.